



PRUEBAS DE UNIDAD EN PYTHON

LABORATORIO DE PROGRAMACIÓN Y LENGUAJES 2026

Errores!!!!!!!



Therac-25 En ciertas condiciones emitía 100 más radiación de la normal. Overflow de software que causó 4 muertes.

Vuelo 140 de Air China
Un error de software desactivo el piloto automático 264 muertes



Arian 5
Un caso de overflow desvió la trayectoria.
Pérdidas millonarias.



Boeing 737 Max Major Design Flaws, Not a Software Failure

How the MCAS (Maneuvering Characteristics Augmentation System) works on the 737 MAX



by Mike Mar 17 - edited ...

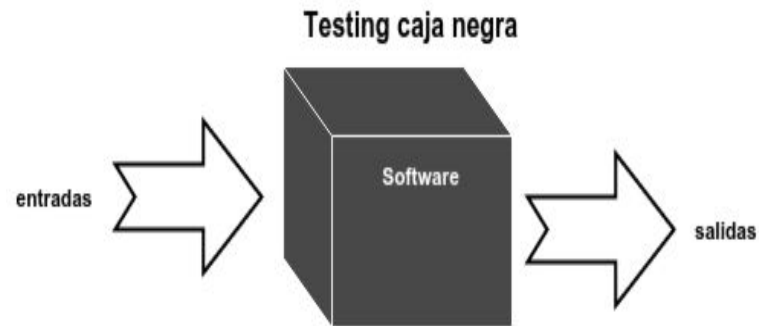
The 737 Max crashes stem from severe design issues and flagrant cost-cutting efforts, not software issues.

Testing de software

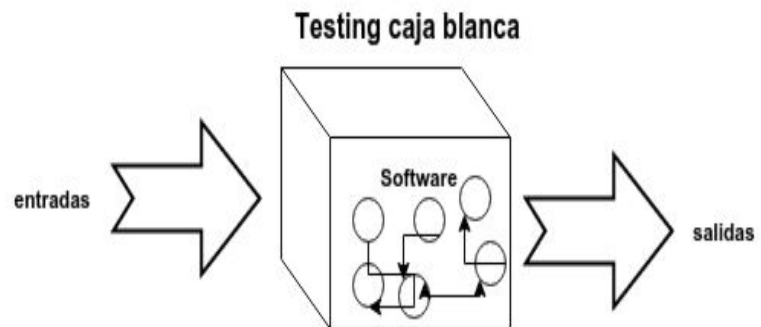
- Es un proceso que busca verificar la exactitud, integridad y calidad de un software.
- Incluye una serie de actividades que buscan encontrar errores y fallas en el software previo a que lleguen al usuario.
- Los errores en el software no identificados a tiempo pueden ser sumamente costosos en términos monetarios o incluso llevar a pérdidas de vidas humanas.

Testing de software

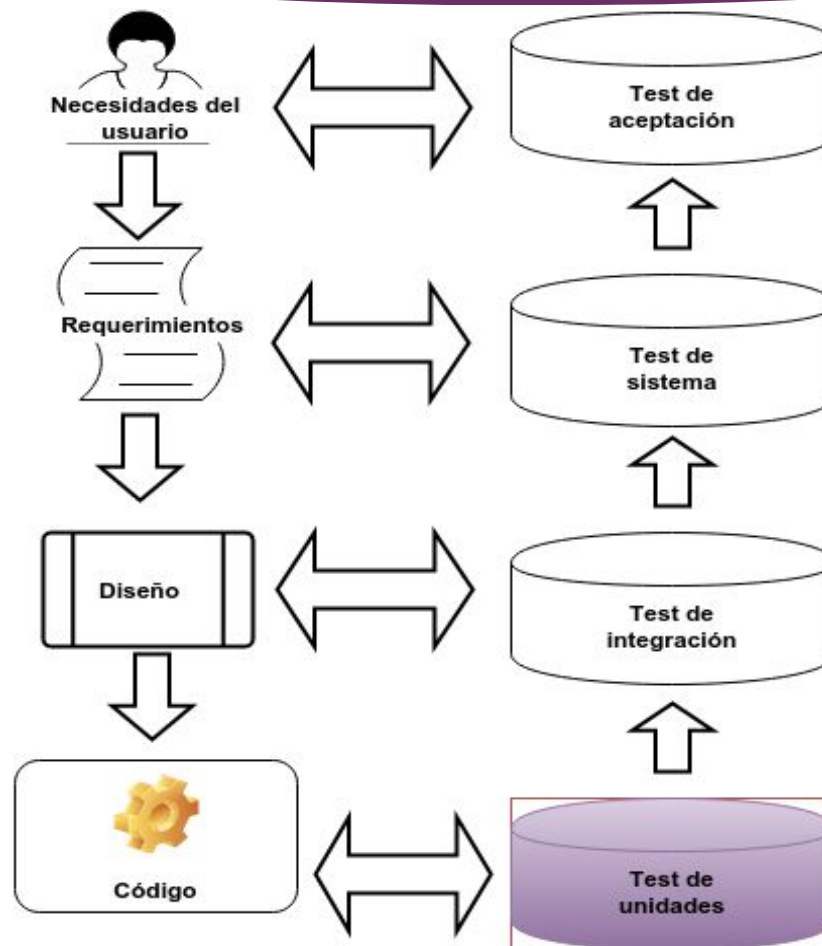
- ▶ Las pruebas se realizan sin conocimiento del funcionamiento interno del software, se preparan un juegos de entradas y se observan si las salidas siguen las especificaciones.



- ▶ Las pruebas se realizan conociendo la estructura interna del software. Los juegos de datos de entrada se seleccionan en base a este conocimiento y suelen apuntar a probar características específicas de su estructura.

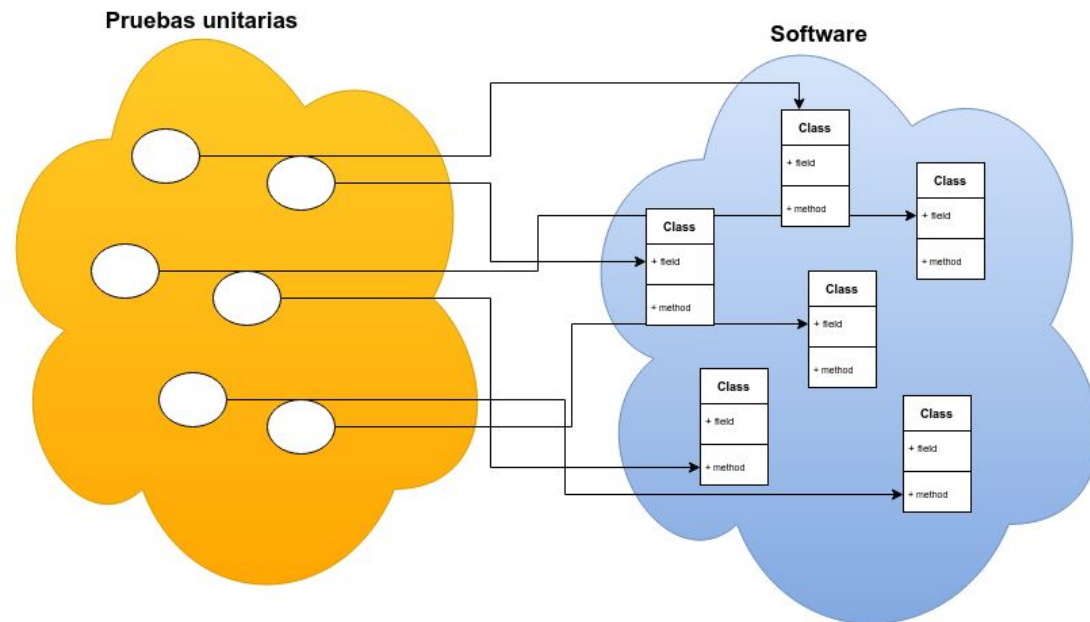


Testing de software - niveles



Test de unidades – Unit Test

- ▶ Son pruebas de bajo nivel (a nivel de código) que se focalizan en una pequeña parte del software. En programación orientada a objetos estas unidades suelen ser la clases.



Python testing – sentencia assert

```
from numbers import Number
def promedio(n):
    assert len(n) != 0
    suma=0
    for i in n:
        assert isinstance(i,Number), f'error de tipos, | no es un numero '
        suma=i+suma
    return suma/len(n)
```

```
>>> promedio([1,4,1,2,3])
2.2
>>> promedio([])
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    promedio([])
  File "C:/Users/matias/Documents/testing/assert.py", line 3, in promedio
    assert len(n) != 0
AssertionError
>>> promedio([1,'a'])
Traceback (most recent call last):
  File "<pyshell#19>", line 1, in <module>
    promedio([1,'a'])
  File "C:/Users/matias/Documents/testing/assert.py", line 6, in promedio
    assert isinstance(i,Number), f'error de tipos, no es un numero '
AssertionError: error de tipos, no es un numero
>>> |
```


Unittest framework

- Python contiene un framework de pruebas unitarias integrado en el api estándar.
- Es un framework de pruebas que forma parte de la familia Xunit.
- Xunit no es un framework sino que se refiere a toda la familia de frameworks que heredan de Sunit el cual fue originalmente escrito para smalltalk por kent beck 1989.
- Este tipo frameworks son fundamentales en la metodología de desarrollo dirigida por pruebas TDD.

Unittest – clase de hoy

- ▶ **En esta clase vamos a recorrer algunas de las abstracciones de python para dar soporte a pruebas unitarias :**
- ▶ **Test case (caso de prueba)**
- ▶ Un caso de prueba es una prueba de unidad individual. Verifica la respuesta de un conjunto de sentencias a ciertos datos de entrada. El framework unittest provee una clase TestCase que puede ser heredada para crear nuevos casos de prueba.
- ▶ **test suite (conjunto de pruebas)**
- ▶ Un test suite es una colección de casos de pruebas y/o otros test suite. Se utiliza para ejecutar pruebas en conjunto..
- ▶ **test runner**
- ▶ Un test runner es un componente que orquesta la ejecución de test y provee una salida al usuario. Puede tener una interfaz gráfica o textual que muestre el resultado de la ejecución de un conjunto de pruebas.

Ejemplo

```
import unittest
#Nuestra clase TestString prueba métodos de la clase String
#Para poder ser una clase de pruebas dentro de python debe
#heredar de la clase unittest.TestCase.
class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        # assertEquals es un método de la clase TestCase que verifica
        # que dos expresiones sean iguales.
        # en este caso estamos probando el correcto funcionamiento del método
        # que pasa un string a mayúsculas
        self.assertEqual('algo'.upper(), 'ALGO')

    def test_isupper(self):
        #assertTrue verifica que una expresión de como resultado True.
        self.assertTrue('ALGO'.isupper())
        #assertFalse verifica que una expresión de como resultado False.
        self.assertFalse('Algo'.isupper())

    def test_split(self):
        s = 'Hola Mundo'
        #probamos que el resultado del método split sea el correcto.
        self.assertEqual(s.split(), ['Hola', 'Mundo'])

if __name__ == '__main__':
    unittest.main()
```

Ejemplo – ejecución

```
...  
-----  
Ran 3 tests in 0.011s  
  
OK  
>>> |
```

Si ejecutamos el código del ejemplo el intérprete nos muestra el resultado de la ejecución exitosa de los 3 casos de test

```
.F.  
=====  
FAIL: test_split (__main__.TestStringMethods)  
-----  
Traceback (most recent call last):  
  File "C:/Users/matias/Documents/testing/TestStringMethods.py", line 23, in test_split  
    self.assertEqual(s.split(), ['Hola1', 'Mundo'])  
AssertionError: Lists differ: ['Hola', 'Mundo'] != ['Hola1', 'Mundo']  
  
First differing element 0:  
'Hola'  
'Hola1'  
  
- ['Hola', 'Mundo']  
+ ['Hola1', 'Mundo']  
?      +  
  
-----  
Ran 3 tests in 0.010s  
  
FAILED (failures=1)
```

Modificando el método de la clase `test_split` de manera que falle vemos que el framework ejecuta todos los casos de prueba y brinda información de los casos que fueron ejecutados con error.

Ejecución de test de línea de comandos

Desde línea de comando podemos ejecutar pruebas contenidas en módulos, clases individuales, o métodos específicos de un TestCase.

```
C:\Users\matias\Documents\testing>python -m unittest TestStringMethods.py
...
-----
Ran 3 tests in 0.001s

OK

C:\Users\matias\Documents\testing>python -m unittest TestStringMethods.TestStringMethods
...
-----
Ran 3 tests in 0.001s

OK

C:\Users\matias\Documents\testing>python -m unittest TestStringMethods.TestStringMethods.test_isupper
.
-----
Ran 1 test in 0.000s

OK
```

- Se puede ejecutar un módulo completo especificando su ruta.
- Si se ejecuta con el modificador “-v” la ejecución muestra un mayor número de detalles.
- Con el modificador “-h” nos muestra todas las opciones disponibles.
- Por último si se ejecuta sin especificar nada inicia un proceso que busca automáticamente los test a partir del path que estamos parados.

La clase TestCase

TestCase es la clase base que debemos usar en nuestras clases de prueba. En esta se encuentra implementada la lógica de pruebas.

Unittest es un framework que sigue la idea de **convención sobre configuración**.

En vez de tener un archivo de configuración en donde definamos donde están las pruebas el framework las encuentra en forma automática siempre y cuando sigamos convenciones.

Para que un método sea una prueba a ejecutar debe empezar de la forma "test_".

```
#heredar de la clase unittest
class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        # assertEquals es un método
        # que compara dos expresiones
        # en este caso estamos
        # que pasa un string a upper
        self.assertEqual('alg', 'alg')
```


TestCase – sentencias assert

La clase TestCase contiene un conjunto de métodos que extienden la sentencia assert de forma conveniente. Cada método assert tiene asociada una excepción que nos permite clasificar el código en categorías.

```
def test_replace(self):  
    s = 'hola mundo'  
    s=s.replace('mundo','marte')  
    x=None  
    #verifica que un elemento se encuentre en una colección  
    self.assertIn('marte', s.split())  
    #verifica que un elemento no se encuentre en una colección  
    self.assertNotIn('mundo',s.split())  
    #verifica que un objeto sea instancia de una clase  
    self.assertIsInstance(s[1],str)  
    #verifica que un objeto no sea instancia de una clase  
    self.assertNotIsInstance(s[1],int)  
    #verifica que un objeto sea None  
    self.assertIsNone(x)  
    #verifica que un objeto no sea None  
    self.assertIsNotNone(s)  
    s=s[1]
```

Python – sentencia with

- La sentencia with fue pensada para mejorar el manejo de excepciones.
- Podemos analizar su uso en base a un ejemplo.

```
#seteo un conjunto de cosas
con.open()
try:
    #realizo tareas que pueden lanzar una excepción
    con.query(query)
finally:
    #realizo tareas que debo realizar incluso
    #si se lanza la excepción
    con.close()
```

```
#seteo un conjunto de cosas
con.open()
try:
    #realizo tareas que pueden lanzar una excepción
    con.query(query)
finally:
    #realizo tareas que debo realizar incluso
    #si se lanza la excepción
    con.close()

class ejecucion_controlada:
    def __enter__(self):
        self.con=database.newCon()
        self.con.open()
        return self
    def __exit__(self):
        con.close()
    def query():
        self.con.query()

with ejecucion_controlada as con:
    con.query(query)

leer_archivo_with("text.txt")
```

- Muchas veces al ejecutar código que puede ser factible de lanzar excepciones necesito realizar tareas para preparar la ejecución.
- Al terminar la ejecución del código requiero realizar un conjunto de tareas para no dejar al sistema en un estado propenso a errores.
- En el ejemplo realizó un query sobre una supuesta conexión de base de datos.
- La sentencia with me permite encapsular el código en una clase que contiene los métodos `__enter__` y `__exit__`. Los objetos de este tipo se llaman contextmanager.
- Al iniciar la ejecución del bloque with el intérprete llama a al método `__enter__` del contextmanager.
- Al terminar el bloque se llama a `__exit__` incluso si se lanzó una excepción.

Python – sentencia with – ejemplo archivos

- La función open de python permite abrir un archivo para su lectura y retorna el manejador de archivos. Open es un objeto contextmanager en ejemplo las dos funciones son equivalentes.

```
def leer_archivo(nombre):  
    f = open(nombre)  
    f.__enter__()  
    print(f.read(1))  
    f.__exit__(None, None, None)  
    print(f.read(1))  
  
def leer_archivo_with(nombre):  
    with open(nombre) as f:  
        print(f.readline())  
  
leer_archivo_with("text.txt")
```

TestCase - Sentencias assert - excepciones

- Cuando codificamos utilizamos excepciones para el manejo de errores esperados e inesperados.
- Nuestras pruebas deben abarcar el código de excepciones verificando que se comporten como se espera.
- La clase TestCase tiene un conjunto de assert orientadas a controlar las excepciones en nuestro código.
- **assertRaises** verifica que se lance una excepción en un bloque with. El test pasa si la excepción es lanzada dentro del with.

TestCase - Sentencias assert - excepciones

```
#assertFalse verifica que una expresion de como resultado False.
self.assertFalse('Algo'.isupper())

def test_split(self):
    s = 'Hola Mundo'
    #Modificamos la sentencia de código para que falle el test.
    self.assertEqual(s.split(), ['Hola', 'Mundo'])
    s = 'Hola, Mundo,marte,luna'
    #string.split(separator, maxsplit) split separa un string utilizando
    #el separador que pongamos como parametro. Por default es el espacio.
    with self.assertRaises(TypeError):
        s.split(2)
```

```
Documents/testing')
....
-----
Ran 4 tests in 0.004s

OK
```

```
#el separador que pongamos como par.
with self.assertRaises(TypeError):
    x=s.split(',')
    print('\n',x[3])
```

```
Documents/testing')
..F.
luna

=====
FAIL: test_split (__main__.TestStringMethods)
-----
Traceback (most recent call last):
  File "C:\Users\matias\Documents\testing\TestStringMethods2.py", line 29, in test_split
    print('\n',x[3])
AssertionError: TypeError not raised

-----
Ran 4 tests in 0.004s

FAILED (failures=1)

In [26]:
```

Sentencias assert - excepciones

Method	Checks that
<code>assertRaises(exc, fun, *args, **kwargs)</code>	<code>fun(*args, **kwargs)</code> raises <i>exc</i>
<code>assertRaisesRegex(exc, r, fun, *args, **kwargs)</code>	<code>fun(*args, **kwargs)</code> raises <i>exc</i> and the message matches regex <i>r</i>
<code>assertWarns(warn, fun, *args, **kwargs)</code>	<code>fun(*args, **kwargs)</code> raises <i>warn</i>
<code>assertWarnsRegex(warn, r, fun, *args, **kwargs)</code>	<code>fun(*args, **kwargs)</code> raises <i>warn</i> and the message matches regex <i>r</i>
<code>assertLogs(logger, level)</code>	The <code>with</code> block logs on <i>logger</i> with minimum <i>level</i>

Sentencias assert

Method	Checks that
<code>assertAlmostEqual(a, b)</code>	<code>round(a-b, 7) == 0</code>
<code>assertNotAlmostEqual(a, b)</code>	<code>round(a-b, 7) != 0</code>
<code>assertGreater(a, b)</code>	<code>a > b</code>
<code>assertGreaterEqual(a, b)</code>	<code>a >= b</code>
<code>assertLess(a, b)</code>	<code>a < b</code>
<code>assertLessEqual(a, b)</code>	<code>a <= b</code>
<code>assertRegex(s, r)</code>	<code>r.search(s)</code>
<code>assertNotRegex(s, r)</code>	<code>not r.search(s)</code>
<code>assertCountEqual(a, b)</code>	<code>a</code> and <code>b</code> have the same elements in the same number, regardless of their order.

stSuite

La clase TestCase - fixtures

- ❖ **setUp():** Este método es llamado antes de la ejecución de cada método test. Aquí se preparan los datos necesarios a utilizar en el test.
- ❖ **tearDown():** Este método es llamado al finalizar la ejecución de cada método y es utilizado para limpiar las instancias utilizadas en el método de prueba. Se ejecuta incluso si una excepción es lanzada durante la ejecución de la prueba.
- ❖ **setUpClass():** este método debe ser implementado como método de clase y se ejecuta al inicio de todas las pruebas de la clase.
- ❖ **tearDownClass():** este método se ejecuta al finalizar todas las pruebas de una clase.

```
@classmethod  
def setUpClass(cls):  
    ...
```

Problema

Realizar la clase agenda que guarde el nombre, apellido, dirección y teléfono de una persona utilizando su dni como referencia. Desarrolle los métodos necesarios para gestionar los datos incluyendo excepciones por datos inválidos. Una vez realizada la clase Agenda escriba la clase AgendaTest que pruebe todos los métodos incluida las excepciones.